

Phase 2 — Common Utilities, Security Middleware & HTTP Enhancements Implementation Report

1. Status

Phase 2 is **COMPLETE**. All security middleware, request validation helpers, query sanitizers, pagination utilities, sensitive data redaction routines, HTTP status & response utilities, and security test suites have been verified with 100% test passing rates and zero regressions across Phase 0 and Phase 1 foundations.

2. Files Created

The following files were created during Phase 2:

- `src/common/http/pagination.ts` — Reusable pagination parsing (`parsePagination`) and metadata calculation (`createPaginationMeta`).
- `src/common/http/query.ts` — Whitelisted sorting, typed query filter parsing, NoSQL operator injection guards, and regex-safe search builders.
- `src/common/http/request-metadata.ts` — Safe extraction of proxy-aware IP, User-Agent, origin, and request metadata.
- `src/common/security/redact.ts` — Deep immutable sensitive data redaction for objects, arrays, strings, and Error objects.
- `src/common/security/index.ts` — Barrel exports for security utilities.
- `src/common/validators/common.validators.ts` — Reusable Zod schemas (`requiredString` , `optionalString` , `positiveInteger` , `nonNegativeInteger` , `booleanCoerce` , `objectIdSchema` , `emailSchema` , `enumSchema` , `paginationQuerySchema` , `validateRequest`).
- `src/common/validators/index.ts` — Barrel exports for validation helpers.
- `docs/PHASE-2-HTTP-SECURITY.md` — Comprehensive technical reference documentation for Phase 2.
- `tests/api/security-middleware.test.ts` — Integration test suite for Helmet, CORS, body limits, request IDs, rate limiting, and routing hardening.
- `tests/unit/pagination.test.ts` — Unit test suite for pagination parsing and metadata calculations.
- `tests/unit/query.test.ts` — Unit test suite for query parsing, sorting, filtering, and safe search.
- `tests/unit/redact.test.ts` — Unit test suite for sensitive data masking and redaction.
- `tests/unit/request-metadata.test.ts` — Unit test suite for safe request metadata extraction.
- `tests/unit/validators.test.ts` — Unit test suite for Zod validation helpers and request validation middleware.

3. Files Modified

- `src/app.ts` — Moved `requestIdMiddleware` to the head of the pipeline to guarantee correlation across all middleware, CORS checks, and error responses; preserved body limits (1MB) and rate limiting.
- `src/config/cors.ts` — Centralized CORS configuration with dynamic whitelist evaluation, credentials support, preflight handling (`maxAge: 86400` , `204`), and safe handling of non-browser/same-origin requests without an `Origin` header.
- `src/config/security.ts` — Hardened Helmet security headers and implemented `createRateLimiter` factory producing standard API error envelopes.
- `src/config/env.ts` — Added `TRUST_PROXY` and `RATE_LIMIT_WINDOW_MS` with strict type coercion and safe defaults.
- `src/config/logger.ts` — Broadened Pino redaction paths with wildcards for sensitive authentication, payment, customer notes, and connection string data.
- `src/common/http/status-codes.ts` — Added `METHOD_NOT_ALLOWED: 405` and `PAYLOAD_TOO_LARGE: 413` .

- `src/common/http/envelope.ts` — Preserved standard Phase 0 response envelopes with helpers for success, created, no-content, and error responses.
- `src/common/http/index.ts` — Exported pagination, query, and request-metadata utilities.
- `src/common/middleware/request-id.middleware.ts` — Enforced strict regex validation (`/^[a-zA-Z0-9_-]{1,128}$/`) on incoming client `X-Request-Id` headers.
- `src/common/middleware/error.middleware.ts` — Added handlers for payload size exceeded (413 `PAYLOAD_TOO_LARGE`) and `URIError` (400 `BAD_REQUEST`), guaranteeing no stack traces or credentials leak.
- `src/common/index.ts` — Exported common security and validator submodules.
- `tests/unit/response.test.ts` — Added unit tests for `sendSuccess` , `sendCreated` , `sendNoContent` , and `sendError` .
- `README.md` — Linked Phase 2 technical documentation.

4. Dependencies

Zero new dependencies were added. Existing dependencies (`helmet` , `cors` , `express-rate-limit` , `cookie-parser` , `zod` , `pino` , `pino-http`) were leveraged completely.

5. Security Middleware

- **Helmet:** Configured with `nosniff` , `SAMEORIGIN` frameguard, `off` for DNS prefetch, suppression of `X-Powered-By` , and environment-aware CSP (disabled in development to support OpenAPI/Swagger, enabled in production).
- **CORS:** Evaluates `ALLOWED_ORIGINS` and `PUBLIC_APP_ORIGIN` dynamically. Rejects disallowed cross-origin browser requests with 403 `FORBIDDEN` standard error envelopes. Allows non-browser/same-origin requests (e.g. curl, health checks). Supports credentials for refresh cookies (`credentials: true`), exposes `X-Request-Id` , and caches preflight `OPTIONS` requests for 24 hours (`maxAge: 86400`).
- **Body Limits:** Enforced at 1MB caps on both `express.json({ limit: '1mb' })` and `express.urlencoded({ extended: true, limit: '1mb' })` . Oversized payloads are caught by `errorHandlerMiddleware` as HTTP 413 `PAYLOAD_TOO_LARGE` .
- **Request IDs:** Incoming `X-Request-Id` values are sanitized against `/^[a-zA-Z0-9_-]{1,128}$/` . Unsafe or missing IDs are replaced with cryptographically secure `req.<uuid>` tokens. Attached to `req.id` , emitted via `X-Request-Id` response header, and embedded in all logs, success metadata, and error response envelopes.
- **Rate Limiting:** In-memory rate limiting powered by `express-rate-limit` . Standard public limiter set to 100 req/min (configurable via `RATE_LIMIT_PUBLIC_PER_MINUTE`). Emits `RateLimit-*` and `Retry-After` headers and returns standard 429 `RATE_LIMITED` envelopes upon exhaustion.

6. HTTP Utilities

- **Pagination:** `parsePagination(query, options)` safely parses and bounds `page` (default 1) and `limit` (default 20, max 100), computing `skip` . `createPaginationMeta({ page, limit, total, nextCursor })` generates standard metadata: { `page` , `limit` , `total` , `totalPages` , `hasNextPage` , `hasPreviousPage` } .
- **Query Parsing:** `containsMongoOperator` and `assertNoMongoOperators` inspect inputs to block raw MongoDB injection (`$where` , `$regex` , `$gt` , `$expr` , `$function` , etc.).
- **Sorting:** `parseSort(query, { allowedFields, defaultSort })` parses sort parameters (e.g. `?sort=-createdAt,price`), strictly rejects or ignores non-whitelisted fields, and emits MongoDB sort criteria (e.g. { `createdAt: -1` , `price: 1` }).
- **Filtering:** `parseFilters(query, { allowed, strict })` whitelists allowable query filters with type coercion and validation (`boolean` , `number` , `objectId` , `date` , `enum` , `string`), prohibiting arbitrary nested object injection.
- **Search:** `sanitizeSearchString(term, maxlen)` trims inputs, normalizes empty values to `null` , and enforces a length cap. `escapeRegex(text)` escapes regex metacharacters, and `buildSafeRegexSearch(term, fields)` produces safe MongoDB `$or` queries.

- **Validation:** Reusable Zod schemas for primitives (`requiredString` , `optionalString` , `positiveInteger` , `nonNegativeInteger` , `booleanCoerce` , `objectIdSchema` , `emailSchema` , `enumSchema` , `paginationQuerySchema`) and `validateRequest({ body, query, params })` middleware.
- **Request Metadata:** `getRequestMetadata(req)` safely extracts proxy-aware client IP, User-Agent, method, URL path, origin, and timestamp while stripping tokens, cookies, and secrets.

7. Sensitive Data Protection

- `redactSensitiveData` traverses nested objects, arrays, and Error instances immutably.
- Protects case-insensitive matching fields (`password` , `passwordHash` , `token` , `accessToken` , `refreshToken` , `resetToken` , `authorization` , `cookie` , `secret` , `apiKey` , `clientSecret` , `smtpPassword` , `cloudinaryApiSecret` , `cvv` , `cardNumber` , `paymentProof` , `mongoUri` , etc.).
- Handles circular references safely (`[Circular]`).
- String-level regex masking (`redactString`) redacts credentials in MongoDB URIs and Bearer tokens.
- Pino HTTP and logger configurations redact sensitive headers (`req.headers.authorization` , `req.headers.cookie` , `x-guest-token`) and sensitive body paths. Full request bodies and authorization headers are never logged.

8. Error Handling

- Centralized `errorHandlerMiddleware` maps domain `AppError` instances, Zod errors, malformed JSON `SyntaxError` (400), payload limits (413), and `URIError` (400) to standard error envelopes.
- Unhandled server exceptions (500) log full stack traces server-side with request ID correlation, but return a client-safe response:

```
{
  "success": false,
  "error": {
    "code": "INTERNAL_ERROR",
    "message": "An unexpected internal error occurred.",
    "details": null
  },
  "requestId": "req_...",
  "meta": { "requestId": "req_...", "timestamp": "..." }
}
```

- No stack traces, database credentials, connection URIs, file paths, or internal query representations reach clients.

9. Environment Configuration

Phase 2 introduced the following environment variables, added to `.env.example` and parsed in `src/config/env.ts` :

Variable	Type	Default	Description
TRUST_PROXY	boolean number string	1	Express reverse proxy trust level (for Nginx/Hostinger)
RATE_LIMIT_WINDOW_MS	number	60000	Rate limiting evaluation window in milliseconds (1 minute)

10. Tests

All automated test suites execute locally using `mongodb-memory-server` and in-memory mocks without external dependencies:

Test File	Test Suite Focus	Test Count
<code>tests/api/security-middleware.test.ts</code>	Request ID, CORS, body limits, Helmet, rate limiting, routing hardening	18
<code>tests/unit/pagination.test.ts</code>	Pagination parsing, limits, skip, and metadata builder	10
<code>tests/unit/query.test.ts</code>	Sort parsing, filter parsing, operator detection, safe regex search	19
<code>tests/unit/redact.test.ts</code>	Sensitive field redaction, nested immutability, circular references, errors	11
<code>tests/unit/validators.test.ts</code>	Zod validation primitives and <code>validateRequest</code> middleware	19
<code>tests/unit/request-metadata.test.ts</code>	Client IP extraction, User-Agent, safe metadata summary	5
<code>tests/unit/response.test.ts</code>	Envelope formatting and <code>sendSuccess/sendCreated/sendNoContent/sendError</code> helpers	6
<code>tests/unit/env.test.ts</code>	Environment parsing, defaults, production refinement	11
<code>tests/unit/errors.test.ts</code>	<code>AppError</code> hierarchy, status codes, error details	10
<code>tests/unit/db-errors.test.ts</code>	MongoDB error normalization and classification	7
<code>tests/unit/db-validators.test.ts</code>	Database custom validators (EGP minor units, <code>ObjectIds</code> , etc.)	8
<code>tests/unit/migrations.test.ts</code>	Migration runner, locking, and status tracking	6
<code>tests/unit/seeds.test.ts</code>	Seed runner, idempotency, and status checks	5
<code>tests/api/app.test.ts</code>	App routing, request ID propagation, 404 handler	8
<code>tests/integration/health.test.ts</code>	<code>/health</code> , <code>/health/live</code> , <code>/health/ready</code> probes	5
<code>tests/integration/app-bootstrap.test.ts</code>	Full HTTP server bootstrap and graceful shutdown	3
<code>tests/integration/db-connection.test.ts</code>	MongoDB connection lifecycle and reconnection events	4
Total	17 Test Suites	155 Tests

11. Verification Results

Command	Result	Details
npm run typecheck	PASS	tsc --noEmit completed with 0 errors
npm run lint	PASS	ESLint completed with 0 errors and 0 warnings
npm run build	PASS	TypeScript compiled cleanly to dist/
npm test	PASS	17/17 test suites passed, 155/155 tests passed (100%)
npm run dev	PASS	Dev server started on port 3000, connected to database, live/ready probes responsive

Live Verification Proof

1. Liveness Probe:

```
curl -i http://localhost:3000/health/live
# HTTP/1.1 200 OK
# X-Request-Id: req_8954daa5-efca-4b4d-a05c-8f8fd14dcc54
# X-Content-Type-Options: nosniff
# X-Frame-Options: SAMEORIGIN
# RateLimit-Limit: 100, RateLimit-Remaining: 99
# Body: {"success":true,"data":{"status":"ok"},"requestId":"req_...", "meta":{"..."}}
```

2. Readiness Probe:

```
curl -i http://localhost:3000/health/ready
# HTTP/1.1 200 OK
# Body: {"success":true,"data":{"status":"ready","database":"connected"}, ...}
```

3. CORS & Custom Request ID:

```
curl -i -H "Origin: http://localhost:4200" -H "X-Request-Id: test-client-req-999" http://localhost:3000/api/v1/health
# HTTP/1.1 200 OK
# Access-Control-Allow-Origin: http://localhost:4200
# X-Request-Id: test-client-req-999
```



4. Disallowed Origin Rejection:

```
curl -i -H "Origin: http://malicious-attacker.com" http://localhost:3000/api/v1/health
# HTTP/1.1 403 Forbidden
# X-Request-Id: req_fd7accbd-9ee6-4f6b-b6cb-1f8f497e512c
# Body: {"success":false,"error":{"code":"FORBIDDEN","message":"CORS origin not allowed..."},"requestId":"req_..."}
```



12. Database Safety Confirmation

- No business collection was modified.
- No business migration was added.
- No fake seed data was created.

- **No destructive database operations (`dropDatabase()` , `drop()` , `deleteMany()` , `updateMany()`) were executed against the real database.**
- All tests ran exclusively against isolated `MongoMemoryServer` instances or mock fixtures.

13. Remaining Issues

None. All verification commands, TypeScript typechecks, ESLint rules, and regression tests passed without errors or warnings.

14. Next Phase Recommendation

Phase 2 is fully complete. Per the Critical Stop Rule, execution is halted here. Ready for **Phase 3 (Authentication, Identity & Session Management)** when authorized.